Horizontal Scaling

add more servers instead of a bigger one

Watch the 3-min video

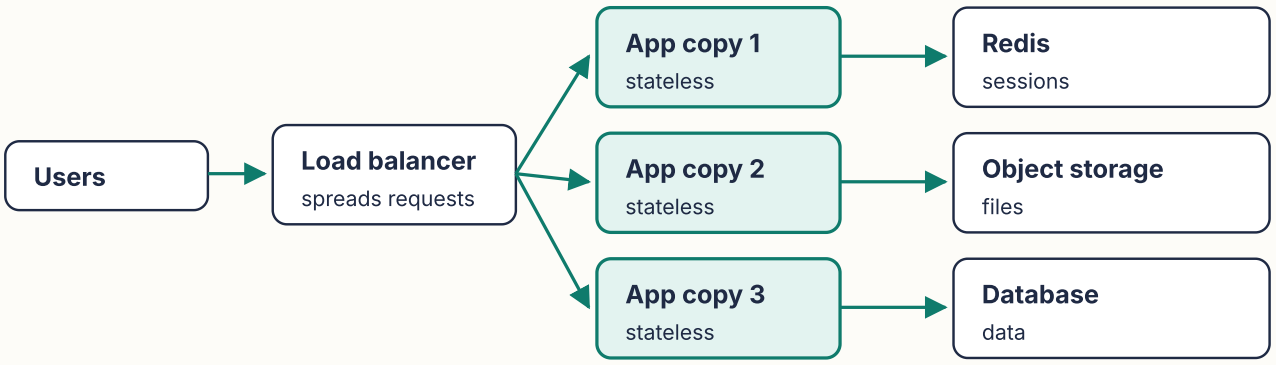
In one line: horizontal scaling (scaling out) runs many copies behind a load balancer; it only works when the service is stateless, and it's much harder for databases because they hold state.

1 Scale out, not up

One machine always hits a ceiling. Scaling out runs many copies of a service side by side, with a load balancer in front spreading requests across them. Need more capacity? Add a copy. Traffic drops? Remove one. There's no hard ceiling, and losing one copy doesn't take the service down.

2 The rule: be stateless

Imagine a server keeps a user's session in its own memory. The next request lands on a different server, the session isn't there, and the user is logged out. The fix is to move state out of the servers: sessions to a shared store like Redis (or a signed token), files to object storage, data to the database.



state lives outside the copies, so any copy can serve any request

Figure 1. A horizontally scaled service: identical, stateless copies behind a load balancer, with state in shared stores.

3 A real example: autoscaling on Kubernetes

ELASTIC FASTAPI SERVICE

Twenty FastAPI pods run behind a Kubernetes Service. A Horizontal Pod Autoscaler (HPA) watches average CPU, targeting, say, 60%. When traffic spikes and CPU rises, it adds pods; when traffic falls, it removes them (with a cooldown so it doesn't flap). This ability to grow and shrink with demand is **elasticity**.

4 What changes when you scale out

	Effect
Availability ↑	One copy dies, the others keep serving (if health checks remove it quickly)
Cost flexible	Pay for what you run; scale down at night
Complexity ↑	Load balancing, health checks, service discovery, config and deploys across many machines
Debugging harder	A request may touch any copy; you need central logs and tracing

5 Why databases are harder to scale out

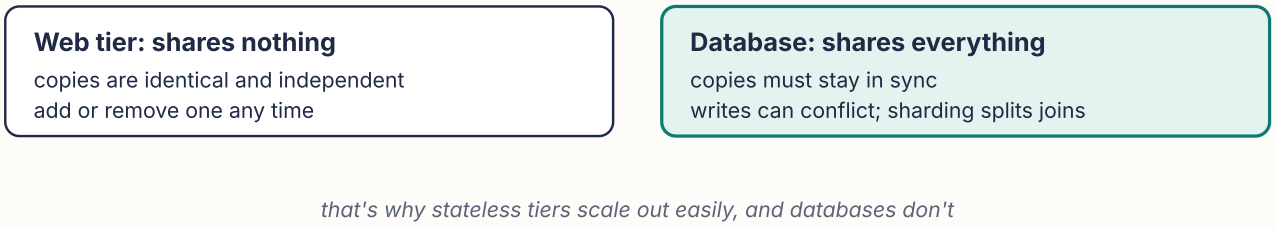


Figure 2. Stateless web servers share nothing. Databases share everything.

Read load can scale with **read replicas** (copies that follow the primary). Write load needs **sharding**: splitting data across machines by a key. Then queries that join across shards, and transactions that touch several shards, become slow or complicated. That's why teams scale the database up first, and shard only when they must.

COMMON MISTAKES

- Scaling out a service that keeps sessions in memory.
- Relying on sticky sessions as the fix (uneven load, lost sessions on failure).
- Autoscaling on CPU when the real bottleneck is the database.
- Forgetting the database connection limit when adding 50 app copies.

6 Key terms

Horizontal scaling: more machines (scale out)

Elasticity: grow and shrink with demand

Read replica: a copy for reads

Stateless: no user state kept between requests

HPA: Kubernetes autoscaler for pods

Sharding: splitting data across machines by a key

7 Interview questions

Q1. What must be true before a service can scale horizontally? BASIC

It must be stateless: sessions, files and data live in shared stores, so any instance can serve any request and instances can be added or removed freely.

Q2. Why is scaling a database horizontally harder? BASIC

State must be replicated (kept in sync) or partitioned (sharded) while staying consistent; cross-shard joins and transactions become hard. Web servers share nothing.

Q3. What is elasticity? BASIC

Adding capacity when load rises and removing it when load falls, automatically, so you pay for what you use.

Q4. Users get logged out randomly after you add servers. Why, and how do you fix it? SCENARIO

Sessions are stored in each server's memory, so a request on another server finds none. Move sessions to Redis or use signed tokens; sticky sessions only hide the problem.

Q5. You scale the API from 10 to 50 pods and the database falls over. What happened? SCENARIO

Each pod opens its own connection pool, so connections jumped 5× past the database's limit. Use a connection pooler (e.g. PgBouncer), smaller per-pod pools, and cache reads.

Q6. Which metric would you autoscale a queue worker on? SCENARIO

Queue depth or message age, not CPU: they measure the backlog directly. Workers waiting on I/O can have low CPU while the queue grows.

Q7. Are sticky sessions a good way to scale a stateful app? TRICKY

Only as a short-term patch: load becomes uneven, sessions are lost when a server dies, and deploys get harder. Externalise the state instead.

Q8. Can you scale writes with read replicas? TRICKY

No. Replicas copy the primary's writes, so every write still goes through the primary. Write scaling needs sharding or a database built to partition writes.

Next: SD-A-008 · Latency backendroadmap.com